



Recibe una cálida:

¡Bienvenida!

Te estábamos esperando 😊 +



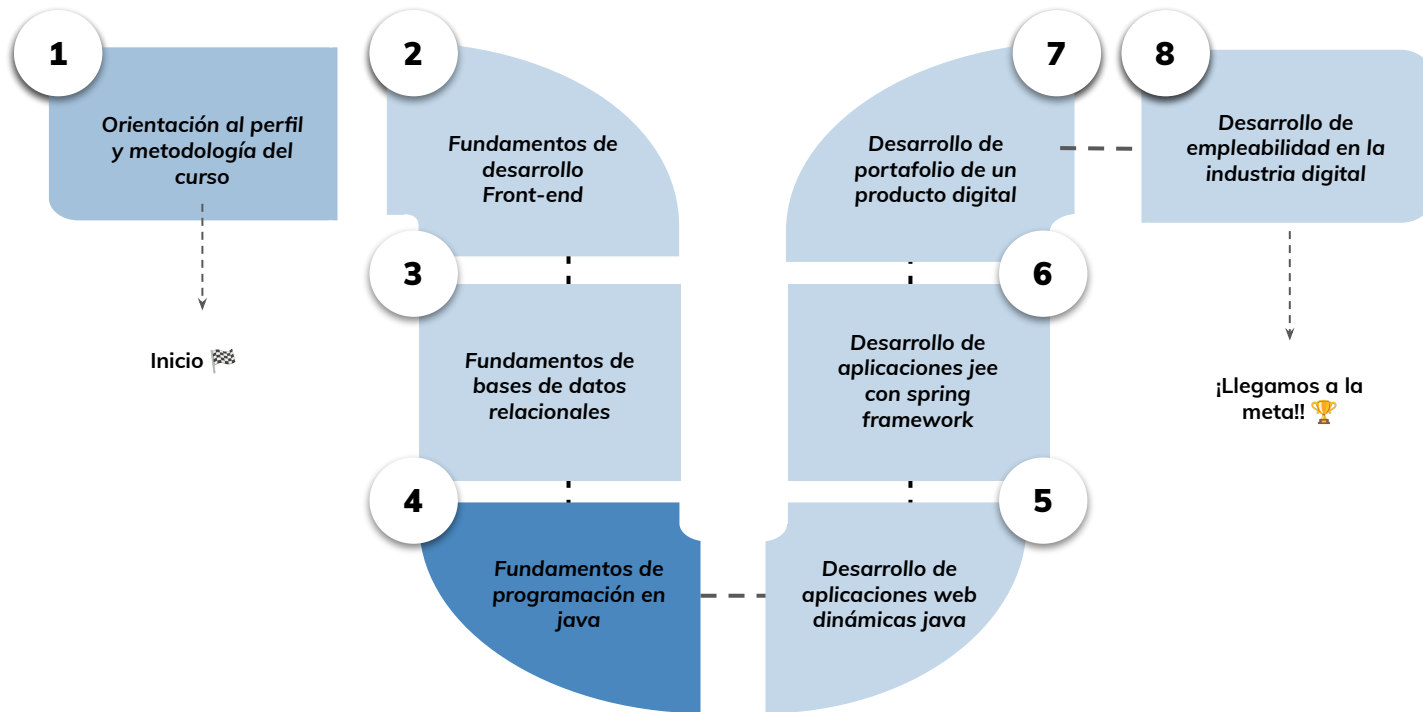
Pruebas unitarias en java-

› Parte 1

Aprendizaje Esperado 7: Implementar una suite de pruebas unitarias en lenguaje JAVA utilizando JUnit para asegurar el buen funcionamiento de una pieza de software.

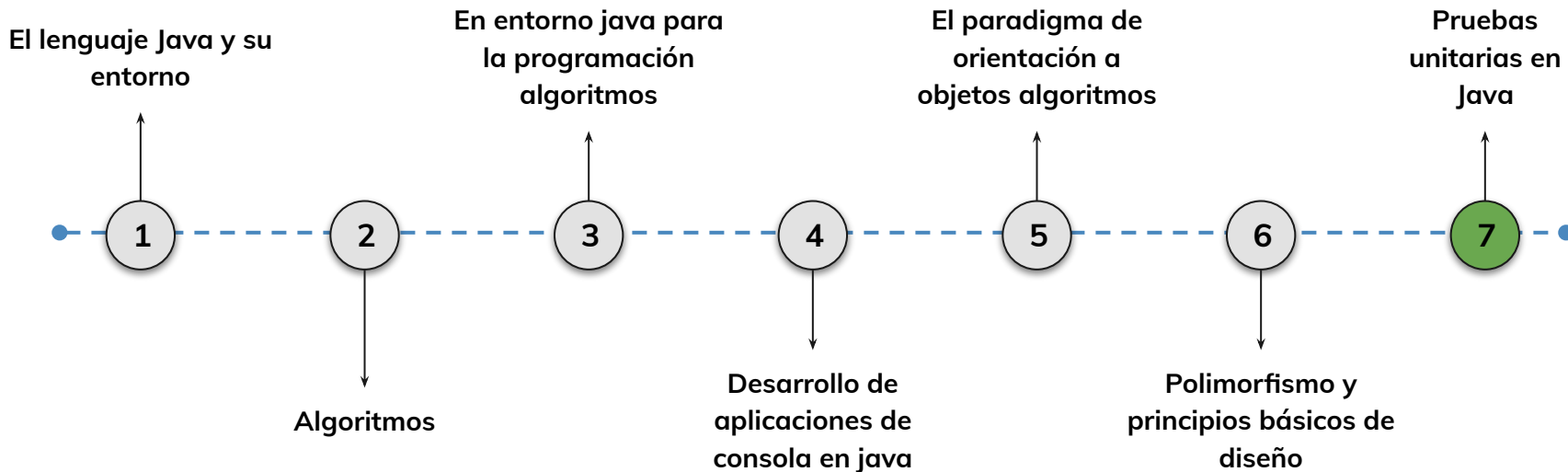
Hoja de ruta

¿Cuáles skills conforman el programa? **Desarrollo de Aplicaciones Full Stack Java Trainee**



Roadmap de lecciones

¿Cuáles **lecciones** estaremos estudiando en este módulo?



Repaso clase anterior

¿Quedó alguna duda?

En la clase anterior :

- Comprendimos los principios fundamentales de diseño orientado a objetos y su impacto en el desarrollo de software.
- Aplicamos técnicas de cohesión y bajo acoplamiento en el diseño de clases.
- Reconocimos y aplicamos los cinco principios SOLID en el desarrollo con Java.
- Refactorizamos código para mejorar la mantenibilidad y la reutilización.
- Desarrollamos software más robusto, flexible y escalable siguiendo buenas prácticas de diseño.



Objetivos de aprendizaje

¿Qué aprenderás?



- Comprender qué son los test unitarios y su importancia.
- Identificar las características, ventajas y desventajas de los tests.
- Reconocer el funcionamiento del framework JUnit.
- Aplicar anotaciones y aserciones en pruebas unitarias.
- Crear clases de prueba en Java usando Eclipse.

Learning Path

¿Cuáles temas trabajaremos hoy?

7.

Pruebas unitarias en Java

Aprenderás a implementar pruebas unitarias utilizando el framework JUnit, entendiendo sus ventajas, estructura, ciclo de vida, buenas prácticas y su integración en IDEs como Eclipse.

Introducción a Test Unitarios

¿Qué son los Test Unitarios?

Características, ventajas y desventajas

JUnit y Eclipse

Introducción a JUnit, ciclo de vida y anotaciones

Integración con Eclipse y generación de casos de prueba

Pruebas unitarias en java

Las pruebas unitarias en Java **son pequeños tests que nos ayudan a comprobar que cada parte del código** (por ejemplo, un método o una clase) funciona correctamente por separado 🧩✅.

Esto nos permite encontrar errores rápido, evitar que algo se rompa cuando hacemos cambios y mejorar la calidad del código desde el inicio.

En esta capacitación vamos **a aprender a crear tests usando JUnit**, entendiendo anotaciones como `@Test` y cómo preparar los datos antes y después de cada prueba. También veremos cómo usar mocks para simular dependencias cuando nuestro código necesita comunicarse con otros componentes.

El objetivo es que puedas asegurarte de que tu código sea confiable, fácil de mantener y seguro para evolucionar, aplicando buenas prácticas desde el primer día 💪👩💻.

» Test Unitarios:

Características

< > Test Unitarios

¿Qué son?:

Los test unitarios en Java son una parte fundamental de la práctica de pruebas en el desarrollo de software. Son **pequeños fragmentos de código diseñados para evaluar el comportamiento de una unidad de código aislada**, por lo general una función o método, en un entorno controlado. Estas pruebas verifican si la unidad de código (como una clase o método) se comporta como se espera.



< > Test Unitarios

Características principales

- **Independencia** : Las pruebas unitarias deben ser independientes entre sí, lo que significa que cada prueba debe poder ejecutarse de forma aislada sin depender de otras pruebas o del entorno externo.
- **Automatización** : Las pruebas unitarias se diseñan para ser ejecutadas de forma automatizada. Se utilizan frameworks como JUnit, TestNG o Mockito para facilitar la creación y ejecución de las pruebas.
- **Aislamiento** : Cada prueba unitaria debe probar una única funcionalidad o comportamiento específico. Para lograr esto, se utilizan mocks o stubs para aislar la unidad en prueba de sus dependencias externas.



< > Test Unitarios

Características principales

- **Cobertura o Coverage** : Las pruebas unitarias deben cubrir la mayor cantidad de código posible. El objetivo es asegurar que todas las funcionalidades y caminos del código sean probados, minimizando la posibilidad de errores.
- **Repetitividad** : Las pruebas unitarias deben ser repetibles y producir resultados consistentes. Esto significa que deben poder ejecutarse en cualquier entorno sin depender de factores externos.
- **Rapidez** : Las pruebas unitarias deben ser rápidas de ejecutar, ya que se espera que se puedan ejecutar frecuentemente, incluso durante el proceso de desarrollo.
- **Legibilidad** : Es importante que las pruebas unitarias sean claras y comprensibles para facilitar su mantenimiento y comprensión por parte de otros desarrolladores.



< > Test Unitarios: Ventajas

Las principales ventajas de implementar test unitarios en nuestros programas son:

- **Detección temprana de errores** : Las pruebas unitarias te permiten identificar errores y problemas en el código de manera temprana. Al ejecutar las pruebas de forma regular, puedes capturar y corregir los errores en las primeras etapas del desarrollo, lo que facilita la solución de problemas y evita que se propaguen a otras partes del código.
- **Mejora de la calidad del código** : Las pruebas unitarias fomentan el desarrollo de un código de alta calidad. Al escribir pruebas, debes pensar en la funcionalidad de forma aislada y en cómo probarla adecuadamente. Esto conduce a un código más modular, desacoplado y mantenible.

Programmer's Problem



< > Test Unitarios: Ventajas

Las principales ventajas de implementar test unitarios en nuestros programas son:

- **Facilita el proceso de refactorización** : Las pruebas unitarias actúan como una red de seguridad al realizar refactorizaciones en el código. Siempre que realices cambios en tu código, puedes ejecutar las pruebas unitarias para asegurarte de que las modificaciones no hayan introducido nuevos errores.
- **Documentación y comprensión del código** : Las pruebas unitarias bien escritas sirven como documentación viva de cómo se espera que funcione el código. Al leer las pruebas, otros desarrolladores pueden comprender rápidamente el comportamiento esperado de una determinada funcionalidad o clase.

Me: I'll add unit tests later

App: *breaks*

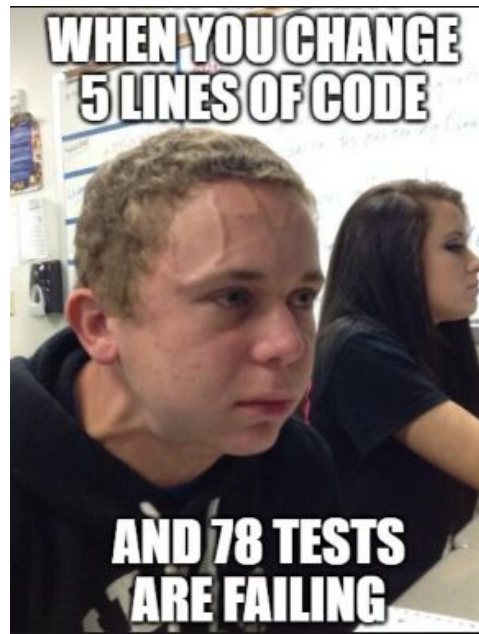
Me:



< > Test Unitarios: Ventajas

Las principales ventajas de implementar test unitarios en nuestros programas son:

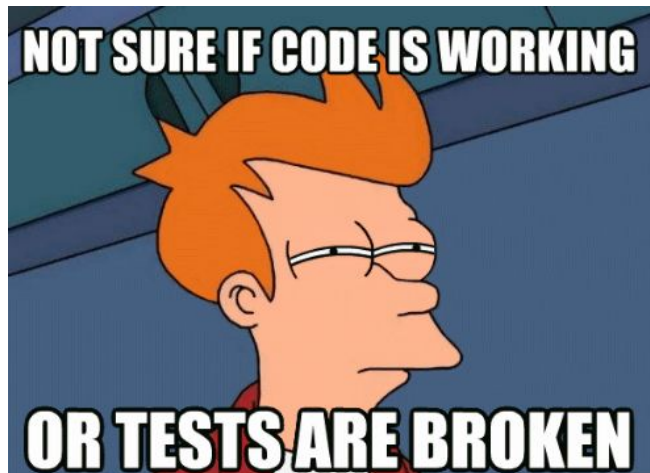
- **Facilita el trabajo en equipo** : Las pruebas unitarias permiten a los desarrolladores trabajar en paralelo y de manera colaborativa. Cada desarrollador puede escribir pruebas para las funcionalidades que está implementando y ejecutarlas para asegurarse de que no haya conflictos con otras partes del código.
- **Mejora la productividad** : Aunque escribir pruebas unitarias requiere tiempo adicional, a largo plazo, puede mejorar la productividad del equipo. Al detectar y corregir errores tempranamente, se reducen los tiempos de depuración y se evitan problemas en etapas posteriores del desarrollo.



< > Test Unitarios: Ventajas

Las principales ventajas de implementar test unitarios en nuestros programas son:

- **Facilita la integración continua y entrega continua** : Las pruebas unitarias son un componente clave de las prácticas de integración continua y entrega continua. Al tener un conjunto de pruebas sólidas, puedes automatizar su ejecución en los sistemas de integración continua, lo que garantiza que el código sea siempre estable y funcional.



< > Test Unitarios: Desventajas

Podemos encontrarnos con distintas escuelas o líneas de pensamiento que expongan “**Desventajas**” de las pruebas unitarias, pero la realidad es que **TODAS** las desventajas, **pueden ser reducidas**, logrando destacar así las ventajas.

Las principales **desventajas** a la hora de implementar Test unitarios en nuestro código y las posibles soluciones son:

- **Costo de desarrollo** : Es necesario invertir tiempo y recursos para escribir las pruebas y mantenerlas actualizadas a medida que el código evoluciona. Esto puede aumentar la carga de trabajo inicial y prolongar los plazos de entrega. Pero sabiendo que si son bien planteadas, otorgan la posibilidad de estabilidad de la app ante futuros cambios.



< > Test Unitarios: Desventajas

- **Curva de aprendizaje** : Para escribir pruebas unitarias efectivas, los desarrolladores deben familiarizarse con los frameworks y herramientas de pruebas, como JUnit o TestNG. Esto implica una curva de aprendizaje adicional para el equipo de desarrollo, especialmente si no están familiarizados con estas herramientas. Si bien es una desventaja, no es nada que no se pueda solucionar estudiando sobre Test Unitarios y practicando en nuestro día a día.
- **Mantenimiento continuo:** A medida que el código evoluciona, las pruebas unitarias también deben actualizarse y mantenerse. Si no se actualizan correctamente, las pruebas pueden volverse obsoletas y generar falsos positivos o falsos negativos. Esto requiere dedicar tiempo y esfuerzo para mantener el conjunto de pruebas actualizado y relevante.



< > Test Unitarios: Desventajas

- **Cobertura limitada** : Aunque las pruebas unitarias pueden cubrir una parte significativa del código, es posible que no logren cubrir todos los casos posibles. Algunos aspectos del sistema, como la interacción con bases de datos o servicios externos, pueden ser difíciles de probar mediante pruebas unitarias, lo que limita la cobertura total.
- **Sobrecarga de pruebas** : Si no se administra adecuadamente, el exceso de pruebas unitarias puede generar una sobrecarga en el proceso de desarrollo. Demasiadas pruebas pueden ralentizar el tiempo de compilación y ejecución, y puede ser difícil mantener todas las pruebas actualizadas y relevantes. Esto puede ser tranquilamente solucionado, planteando bien los diseños y arquitecturas de la app, con el fin de evitar monolitos saturados en múltiples módulos y tests.



< > Desarrollo Dirigido por Pruebas (TDD)

- El Test Driven Development (TDD) o Desarrollo Dirigido por Pruebas es una metodología ágil en la que las pruebas se crean antes del código funcional.

Su objetivo es garantizar que cada parte del software cumpla exactamente con los requisitos definidos y que los errores se detecten lo más temprano posible.

- Ciclo de TDD
 - Red (Escribir la prueba):
Se escribe una prueba unitaria que inicialmente falla, ya que la funcionalidad aún no existe.
 - Green (Implementar el código mínimo):
Se desarrolla el código necesario para que la prueba pase.
 - Refactor (Mejorar el código):
Se refactoriza el código manteniendo las pruebas en verde, asegurando calidad y legibilidad.

```
// Paso 1: Crear test antes del código
@Test
public void testEsMayorDeEdad() {
    assertTrue(ValidadorEdad.esMayorDeEdad(20));
    assertFalse(ValidadorEdad.esMayorDeEdad(15));
}

// Paso 2: Implementar código mínimo
public class ValidadorEdad {
    public static boolean esMayorDeEdad(int edad) {
        return edad >= 18;
    }
}
```



LIVE DEMO

Ejemplo en vivo

Imaginemos el siguiente escenario y respondamos entre todos:

Antes de permitir la creación de una Cuenta Bancaria, debemos crear una función que valide si una persona es mayor de edad según su edad ingresada.

1. ¿Cómo podrían probar que la función funciona correctamente antes de usarla?
2. ¿Qué ventajas les daría crear pruebas unitarias?
3. ¿Pero también habría desventajas o limitaciones?

Tiempo: 20 minutos

➤ Introducción a JUnit

< > JUnit

¿Qué es?:

JUnit es un marco de prueba unitaria para Java que proporciona estructura y herramientas para escribir y ejecutar pruebas unitarias de manera automatizada. JUnit simplifica la creación y ejecución de pruebas, facilitando la identificación temprana de errores y la mejora de la calidad del código.

JUnit es ampliamente utilizado en la comunidad de desarrollo de Java y se ha convertido en una herramienta estándar para la escritura de pruebas unitarias. Permite a los desarrolladores definir casos de prueba, ejecutarlos automáticamente y generar informes detallados sobre el estado de las pruebas.



< > JUnit

Ciclo de Vida de una Prueba :

JUnit sigue un ciclo de vida para las pruebas: **configuración** (setup), **ejecución** de la prueba y **limpieza** (teardown).

La configuración y la limpieza se realizan antes y después de cada prueba para garantizar que estén en un estado conocido.

Mensajes de Error Significativos :

Cuando una prueba falla, JUnit proporciona mensajes de error significativos que ayudan a identificar la causa del fallo.

Esto facilita la depuración y corrección de problemas.



< > JUnit

Herramientas de IDE:

Muchas herramientas de desarrollo integrado (IDE) como Eclipse, IntelliJ IDEA y NetBeans ofrecen soporte integrado para JUnit. Esto facilita la creación, ejecución y análisis de pruebas desde el entorno de desarrollo.

Cobertura de Código:

Algunas herramientas y complementos de JUnit pueden calcular la cobertura de código, lo que te permite saber cuántas líneas de código están siendo probadas por tus pruebas.

Pruebas Continuas (CI/CD):

Las pruebas unitarias con JUnit son esenciales en las prácticas de integración continua (CI) y entrega continua (CD), ya que ayudan a garantizar que el código nuevo no rompa funcionalidades existentes.



< > JUnit

¿Qué es necesario conocer antes de escribir Test con JUnit?

Programación en Java : Es fundamental tener un buen entendimiento de los conceptos básicos de programación en Java, como variables, tipos de datos, estructuras de control (if, bucles, etc.), clases, métodos y excepciones.

POO: Dado que JUnit se utiliza para probar unidades de código a nivel de clase o método, es importante tener conocimientos de programación orientada a objetos.

Método de prueba : Antes de utilizar JUnit, es beneficioso entender qué es un método de prueba y cómo funciona. Un método de prueba es una función que verifica el comportamiento esperado de una unidad de código específica. Debes comprender cómo escribir métodos de prueba y cómo verificar resultados utilizando aserciones (assertions).

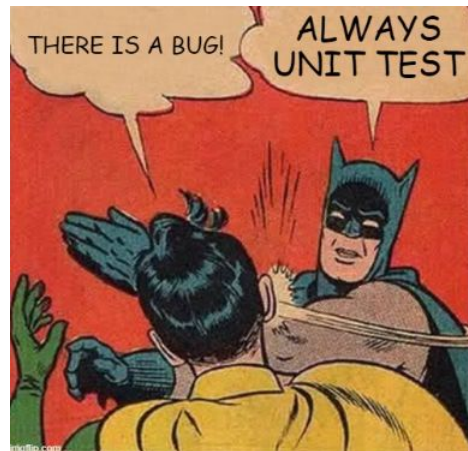


< > JUnit

¿Qué es necesario conocer antes de escribir Test con JUnit?

Anotaciones: JUnit utiliza anotaciones Java para marcar los métodos de prueba y definir su comportamiento. Es importante comprender cómo funcionan las anotaciones en Java y cómo se aplican en el contexto de JUnit. Algunas anotaciones clave en JUnit son **@Test**, **@BeforeEach**, **@AfterEach**, **@BeforeAll** y **@AfterAll**, conceptos que más adelante explicaremos.

Aserciones (assertions) : Las aserciones son declaraciones que se utilizan para verificar si una condición es verdadera o falsa. JUnit5 proporciona un conjunto de métodos de aserción que se utilizan para evaluar y validar resultados en las pruebas unitarias. Es importante conocer los diferentes métodos de aserción disponibles en JUnit5 y cómo utilizarlos correctamente.



LIVE CODING

Ejemplo en vivo

Vamos a crear las primeras clases Test para el proyecto de BilleteraVirtual:

1. Crear una clase de pruebas CuentaTest para probar la clase Cuenta.
2. Crear una clase de pruebas TransaccionTest para probar la clase Transaccion.
3. Crear una clase de pruebas MonedaTest para probar la interfaz Moneda.
4. Explicar el uso de **assertEquals** para verificar el resultado.

Tiempo: 30 minutos

➤ Casos de Prueba

< > Caso de Prueba

¿Qué son?:

Son unidades de trabajo que se utilizan para **verificar el comportamiento de una parte específica de un programa** . Representan situaciones de **entrada , ejecución** y **resultados esperados** de una función o método en el código.

Los casos de prueba son una parte fundamental del proceso de prueba unitaria y se utilizan para asegurarse de que las unidades de código, como clases o métodos, funcionen correctamente.



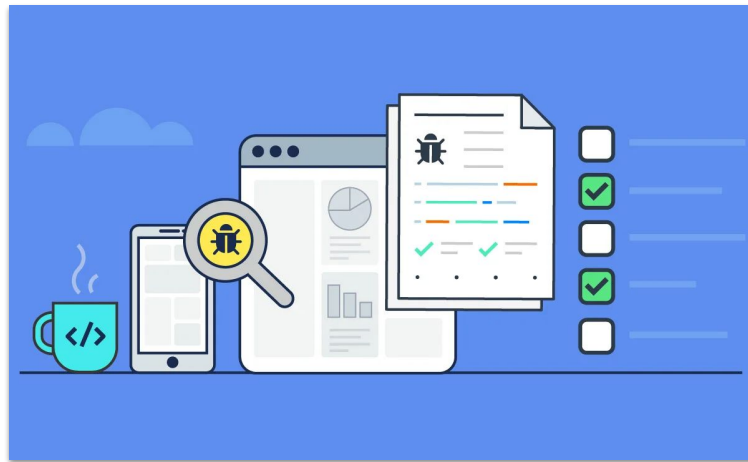
< > Casos de Prueba

Cobertura (coverage):

A la hora de crear los casos de prueba deberemos tener en cuenta que porcentaje de **coverage** o cobertura estableceremos:

- más del 70% es el valor mínimo aceptable
- entre el 85% y 95% es muy bueno
- más del 95% es excelente.

Una vez definido este valor, deberemos **considerar todas las clases a cubrir con test** y qué casos posibles vamos a contemplar en los test.

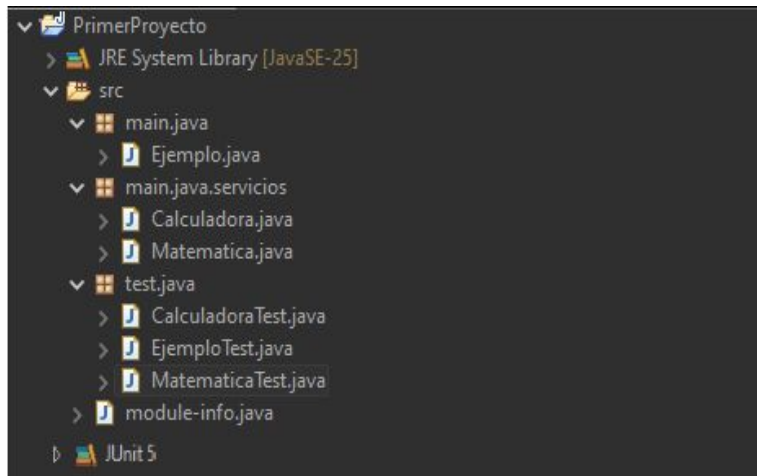


< > Casos de Prueba

Algunos criterios para decidir cómo organizar nuestros tests.

1. **Tantas clases Test como Clases a testear** :
Se debe generar **una ClaseTest por cada Clase** que se desea testear.

Un error común al iniciarse en la creación de los test unitarios es que dentro de una ClaseTest, busquemos testear tanto una Clase1 como una Clase2. Si deseamos testear ambas clases deberemos generar dos Clases Test distintas para cubrirlas.



< > Casos de Prueba

2. **Funciones a testear:** Si la Clase a Testear tiene cinco funciones, la ClaseTest deberá tener un mínimo de cinco @Test. Nunca se debe integrar en un @Test más de una función a testear.

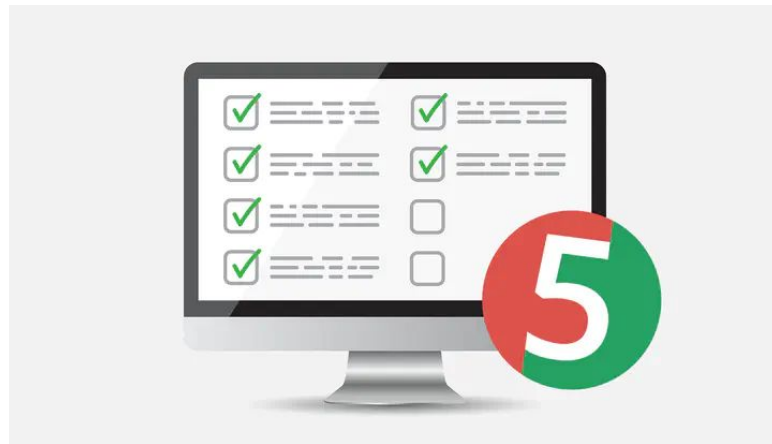
Algo que sí es posible dentro de un @Test es intentar testear dos posibles salidas, un ejemplo de eso sería la siguiente función, parImpar, que puede devolver dos posibles valores (el @Test posee dos assertEquals).

```
public String parImpar(int num) {  
    if (num % 2 == 0) {  
        return "Par";  
    } else {  
        return "Impar";  
    }  
}
```

```
@Test  
void parImpar() {  
    String expected = "Par";  
    String actual = this.matematica.parImpar(2);  
    assertEquals(expected, actual);  
  
    expected = "Impar";  
    actual = this.matematica.parImpar(1);  
    assertEquals(expected, actual);  
}
```

< > Casos de Prueba

3. **Casos de prueba:** Una vez definida la clase que queremos cubrir con los test, se deberá entender que funciones es necesario cubrir, qué parámetros de entrada necesitan las mismas, que parámetros de salida tendrán y qué posibles errores pueden ocurrir al usarlas.



LIVE DEMO

Ejemplo en vivo

Probando: Vamos a realizar algunas pruebas para poner en práctica lo aprendido.

1. *Crear casos de prueba para depositar(), retirar() y getSaldo().*
2. *Explicar cómo se parametrizan las pruebas con diferentes casos y datos de entrada*

Tiempo: 20 minutos

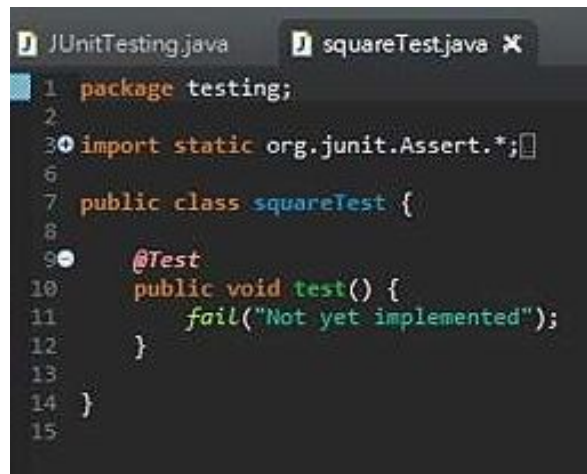
➤ Integración de JUnit con Eclipse

< > Integración de JUnit con Eclipse

Aprovechando las ventajas del IDE:

Una de las principales ventajas de desarrollar java en Eclipse, es que es un IDE con múltiples integraciones, una de ellas es JUnit.

En las últimas versiones de Eclipse y en la mayoría de los casos, no es necesario ningún tipo de integración adicional. Solo deberemos tener en cuenta algunas consideraciones como las que veremos a continuación.

A screenshot of the Eclipse IDE interface. The top bar shows two open files: 'JUnitTesting.java' and 'squareTest.java'. The 'squareTest.java' file is active and displays the following Java code:

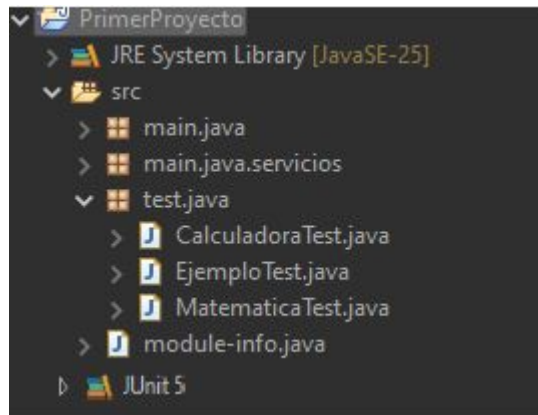
```
1 package testing;
2
3 import static org.junit.Assert.*;
4
5
6
7 public class squareTest {
8
9     @Test
10     public void test() {
11         fail("Not yet implemented");
12     }
13
14 }
15
```



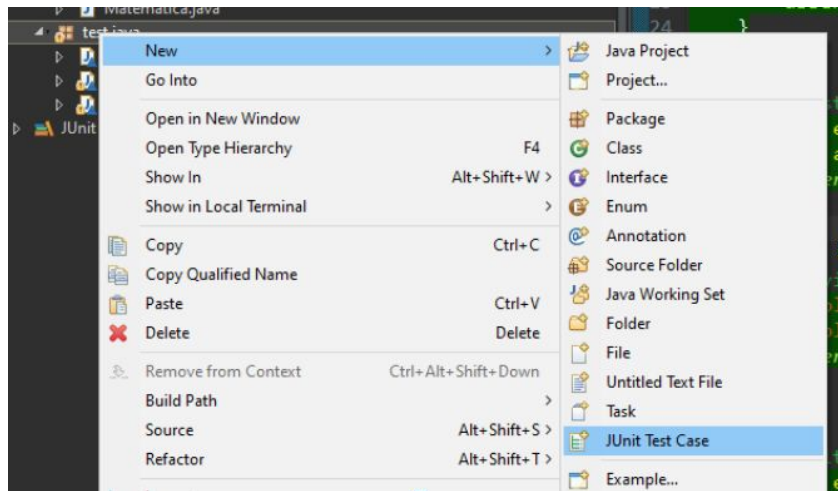
< > Integración de JUnit con Eclipse

A tener en cuenta:

- **Versión de JUnit** : Es recomendable utilizar JUnit 5, pero si se implementa una versión anterior, deberás prestar atención a las @Anotaciones a usar, ya que pueden variar.
- **Gestión de Paquetes** : Es necesario que la estructura del proyecto esté ordenada. Para esto es importante tener los Test en un paquete diferenciado del resto de la app, como podemos ver en la imagen: dentro de main tendremos nuestra App en sí, con su organización de paquetes determinada. Por otro lado, el **paquete test** tendrá un sub paquete llamado java, que es donde se contendrán las clases Test.



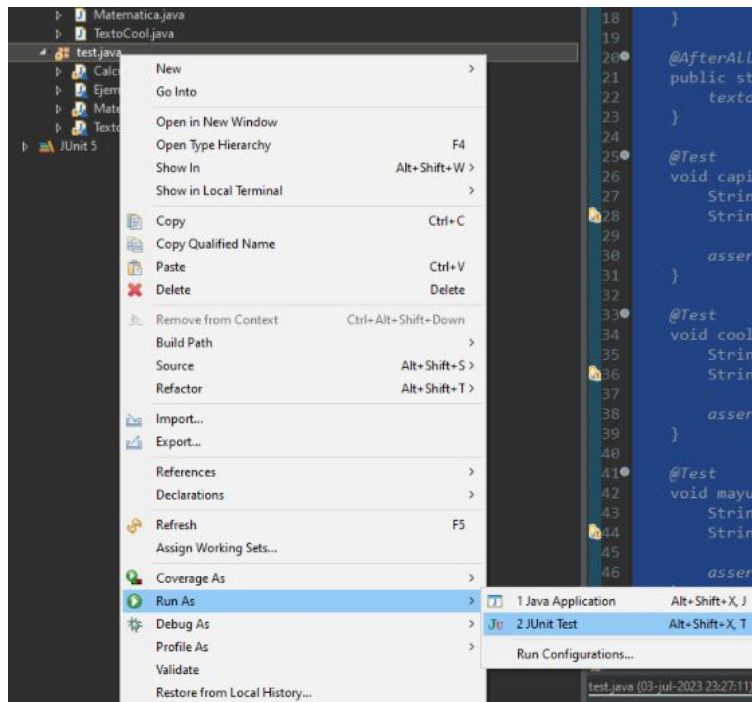
< > Integración de JUnit con Eclipse



A tener en cuenta:

- **Generate Test Cases:** La manera más sencilla de generar un test case o ClaseTest, es haciendo click derecho sobre el paquete donde quiero generarlo y colocarle el nombre correspondiente **[NombreOriginalDeLaClase]+Test**.

< > Integración de JUnit con Eclipse



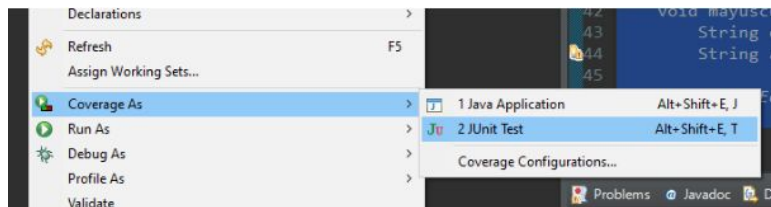
A tener en cuenta:

- **Corriendo los Test:** Para correr todos los test desde Eclipse, puede hacer click derecho sobre el paquete de Test y seleccionar Run As > JUnit Test.

En caso de querer correr los test de una sola clase, realizar el click derecho desde la clase deseada.

< > Integración de JUnit con Eclipse

- **Corriendo los Test:** Si se desea ver el coverage de los test realizados, debes hacer click derecho y seleccionar “**Coverage As**”. Se desplegará la siguiente pantalla.



test.java (03-jul-2023 23:43:19)

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▲ Ejemplo	90,6 %	329	34	363
▲ src	90,6 %	329	34	363
▲ main.java.servicios	83,8 %	155	30	185
▸ Calculadora.java	0,0 %	0	30	30
▸ Matematica.java	100,0 %	36	0	36
▸ TextoCool.java	100,0 %	119	0	119
▲ main.java	0,0 %	0	4	4
▸ Ejemplo.java	0,0 %	0	4	4
▲ test.java	100,0 %	174	0	174
▸ CalculadoraTest.java	100,0 %	4	0	4
▸ EjemploTest.java	100,0 %	25	0	25
▸ MatematicaTest.java	100,0 %	104	0	104
▸ TextoCoolTest.java	100,0 %	41	0	41



LIVE CODING

Ejemplo en vivo

¡Vamos a comenzar la estructura para testear!:

En este ejemplo vamos a crear los paquetes y clases de Test necesarios para algunos ejercicios anteriores que hemos realizado.

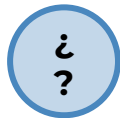
1. Crear un método a evaluar en cada clase Test creada y mostrar cómo correrlo en Eclipse **con** y **sin** porcentaje de cobertura.

Tiempo: 25 minutos

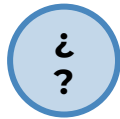
#Momentode Preguntas...



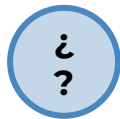
¿Qué diferencia hay entre una clase de pruebas y una clase funcional?



¿Qué ocurre si no mantenemos los test actualizados?



¿Qué beneficios aporta el testing a la calidad del software?



¿Cómo integrarías esto a un flujo de CI/CD?



Momento:

Time-out!



5 -10 min.





Ejercicio N° 1

Validar Edad para Cuenta Bancaria

Validar Edad para Cuenta Bancaria

Contexto: 🙌

Tu equipo está desarrollando una billetera virtual. Antes de permitir la creación de una cuenta bancaria, necesitas validar si una persona es mayor de edad.

Consigna: 📝

Implementa una función en Java que reciba como parámetro la edad de una persona y devuelva si es mayor de edad. Luego, crea una clase de test unitaria utilizando JUnit que verifique al menos dos casos:

1. Persona mayor de edad.
2. Persona menor de edad.

Tiempo 🕒: 40 minutos

Paso a paso: ⚙️

1. Crea una clase ValidadorEdad con un método esMayorDeEdad(int edad).
2. Implementa la lógica para determinar si es mayor a 17 años.
3. Crea la clase ValidadorEdadTest en un paquete separado (test/java).
4. Usa @Test y assertTrue / assertFalse de JUnit.
5. Corre las pruebas desde Eclipse.

¿Alguna consulta?





Resumen

¿Qué logramos en esta clase?



- ✓ Entender qué son las pruebas unitarias y su utilidad.
- ✓ Conocer JUnit y sus anotaciones básicas.
- ✓ Crear una suite de pruebas básicas en Java.
- ✓ Usar Eclipse para generar y correr pruebas.
- ✓ Aplicar buenas prácticas de testing.
- ✓ Valorar la cobertura y organización de los casos de prueba.
- ✓ Enfrentar ventajas y desventajas del testing unitario.



¡Ponte a prueba!

Momento de ejercitación

Te invitamos a aprovechar esta última sección del espacio sincrónico para realizar de manera individual las **actividades disponibles en la plataforma**. Estas propuestas son clave para afianzar lo trabajado y **forman parte obligatoria del recorrido de aprendizaje**.

👉 **Análisis de caso** ————— 👉 **Selección Múltiple**

👉 **Comprensión lectora**

Si al resolverlas surge alguna duda, compartela o tráela al próximo encuentro sincrónico.

< **¡Muchas gracias!** >

